

Name: Hanzla Ahmad

SAP ID: 61002

Course: Game Programming

Submitted to: Sir Hisham Zahid

Assignment 3

Among Us Multiplayer Game Code Architecture Design

Game Overview

Among Us is a small-session social deduction game built around asymmetric information. Crewmates try to complete tasks and identify Impostors, while Impostors try to sabotage systems, kill discreetly, and survive discussion and voting. Public store pages describe the core loop in exactly those terms and also confirm that the game now includes optional roles, multiple maps, and at least the Classic and Hide n Seek modes on Steam.

Those public requirements drive the architecture more than any cosmetic design detail. A game with host-list discovery, private room codes, text chat, cross-platform play, and both online and local Wi-Fi modes needs a clean separation between client presentation, session discovery, transport, authoritative round logic, and profile or cosmetic persistence. In other words, the hard problem is not rendering beans on a map; it is keeping secret information, phase transitions, and player state consistent across devices and connection types.

High-Level Architecture

I would build the game around four runtime zones. The **client application** would own input, UI, animation, audio, and local presentation state. The **online service tier** would own authentication, lobby browsing, room-code joins, quick joins, and session bootstrap. The **authoritative match room** would own the actual game rules for roles, tasks, sabotage, meetings, voting, and win conditions. The **persistence and operations tier** would own profiles, cosmetics, settings, telemetry, analytics, and moderation events. That split lines up well with Unity's documented multiplayer services model, where Lobby, Matchmaker, Relay, and sessions are first-class concepts inside the Multiplayer Services SDK.

Because *Among Us* is primarily a social game rather than a ranked esports, I would make public host lists and private room codes the primary entry points, with quick join as a convenience flow rather than the centre of the product. I would also abstract transport and discovery behind interfaces so the same match logic can work for both local Wi-Fi sessions and internet sessions. That is a better fit for the game's documented support for both LAN-style and online play than hard-wiring one transport into the rules layer.

The diagram below shows that split in one view.

Among Us high-level architecture

The main systems inside that structure would be the Player System, Lobby and Discovery System, Role System, Task System, Sabotage and Combat System, Meeting and Voting System, Game State Management System, Cosmetics and Profile System, and UI System. Official pages matter here: Steam explicitly mentions host lists, text chat, different roles, multiple maps, and cross-platform support, while the account-linking guide shows that player-facing persistence is a real product concern rather than an afterthought.

Code Structure

I would organise the repository by **feature first** and **layer second**. Concretely, that means folders such as Lobby, Match, Roles, Tasks, Sabotage, Meetings, Profiles, and Cosmetics, and inside each of the larger features I would split code into Domain, Application, Infrastructure, and Presentation. That structure keeps ownership clear and prevents lobby code from leaking into round-logic code or UI scripts from mutating match state directly. It also fits Unity's documented component model, where GameObjects are containers for Components, and ScriptableObjects are shared data containers rather than per-instance behaviour blobs.

```
Assets/  
  Scripts/  
    Core/  
      Bootstrap/  
      Configuration/  
      Eventing/  
      Time/  
    Networking/  
      Session/  
      Transport/  
      Replication/  
      Security/  
    Features/  
      Lobby/  
        Domain/  
        Application/  
        Infrastructure/  
        Presentation/  
      Match/  
        Domain/  
        Application/  
        Infrastructure/  
        Presentation/  
      Roles/  
      Tasks/  
      Sabotage/  
      Meetings/  
      Profiles/  
      Cosmetics/  
      UI/
```

Shared/
Contracts/
DTOs/
Utilities/

Inside a live round, I would centre the domain around a `MatchContext` that coordinates a `MatchPhaseController`, `PlayerRegistry`, `RoleAssignmentService`, `TaskSystem`, `SabotageSystem`, `KillSystem`, `MeetingSystem`, `VisionSystem`, and `WinConditionEvaluator`. Each player avatar would be a composed object with focused components such as movement, interaction, vision, vent access, task progress, and role ability components, instead of a deep inheritance tree like `CrewmatePlayer`, `ImpostorPlayer`, `EngineerPlayer`, and so on. Static content such as `RoleDefinition`, `TaskDefinition`, `MapDefinition`, and `SabotageDefinition` would live in `ScriptableObjects` so new roles, task variants, and maps can be added without rewriting the networking core.

Communication between systems would follow three lanes. Inside one process, systems would publish typed domain events such as `BodyReported`, `MeetingStarted`, or `SabotageResolved`. Across the network, clients would send **intent messages** such as `MoveIntent`, `InteractIntent`, `CallMeetingIntent`, and `CastVoteIntent`. Presentation code would then subscribe to projected read models rather than pulling directly from mutable gameplay objects. That arrangement works well with Unity's documented distinction between persistent replicated state and one-off network messages.

The diagram below shows the shape of that code organisation.

Among Us code structure

Networking and State Flow

I would use a **server-authoritative room model** for the actual match. That is the right call for *Among Us* because the game's business value sits in secrecy, fairness, and trusted outcomes, not ultra-low-latency shooter movement. Unity's current multiplayer stack explicitly supports sessions for both peer-to-peer and dedicated server experiences, and its RPC documentation also makes clear that client-to-client calls still route through the server as a proxy rather than forming direct client meshes. On that basis, the cleanest design is: clients send intentions, the room validates them, and only the room commits game truth.

For replication, I would split data exactly the way Unity recommends. **Persistent round state** such as current phase, alive or dead status, task totals, sabotage countdowns, and meeting status should use `NetworkVariable`-style replicated state, because Unity documents `NetworkVariables` as the mechanism for persistent synchronised properties. **Transient moments** such as playing a kill animation, opening the discussion screen, revealing vote results, or flashing a sabotage alarm should use RPCs or equivalent one-off messages, because Unity distinguishes those from persistent replicated state.

The round itself should be controlled by a finite-state machine rather than scattered booleans. The clean flow is Lobby -> RoundInitialisation -> FreePlay <-> Meeting -> Resolution -> EndGame. That fits the official gameplay loop of tasks, sabotage, body reports, emergency meetings, and voting, and it gives the code a brutal but healthy rule: every incoming command is valid only in specific phases. That sharply reduces edge-case bugs and makes testing far easier.

Design Decisions

The core design decision is to treat *Among Us* as a **small, high-trust-risk multiplayer game**. That means I would not optimise first for massive-scale persistence, and I would not optimise first for twitch-grade rollback networking either. I would instead optimise for maintainability, secret-information control, and cheap feature extension. That is why the architecture combines an authoritative room, feature-sliced modules, composition-heavy entity design, and data-driven content definitions. Unity's own documentation supports the underlying building blocks here: components for behaviour composition and ScriptableObjects for shared authored data.

I also think this is the best approach because it keeps future content cheap. A new map mostly becomes a new MapDefinition plus room interactables. A new role mostly becomes a RoleDefinition plus rule policies and UI hooks. A new sabotage mostly becomes a data asset, resolver stations, and a domain rule module. None of those changes should require transport rewrites or a redesign of the base player object. For an assignment focused on software architecture rather than game design detail, that modularity is the strongest argument in favour of this solution.

The trade-off is straightforward: an authoritative room costs more to build and operate than a rough prototype hosted entirely on a client, and information partitioning adds engineering overhead. But that extra spend buys a codebase that is fairer, easier to test, easier to extend, and far less likely to collapse when someone adds a new role, map, mode, or platform. For *Among Us*, that is the right trade.